where does the error come from on line 10 for main.js

Bash Show midterm files and main.js

IN

```
cd /Users/joychang/Documents/web-tool-fa26/midterm && ls && cat -n main.js | head -40 && cat -n *.html
```

OUT

```
index.html
main.js
style.css
     1	console.log("javescript linked")
     2
     3	let userInput = [];
     4
     5	let input = document.querySelector("entries");
     6
     7	input.addEventListener("keypress",(e)=>{
     8	    if(e.key === "Enter"){
     9	        userInput.push(input.value);
    10	})
    11
    12	console.log(userInput);     1	<!doctype html>
     2	<html>
     3	  <head>
     4	    <meta charset="utf-8" />
     5	    <title>ready.blog</title>
     6	    <link rel="stylesheet" href="style.css" />
     7	   <script src="main.js"></script>
     8
     9	  </head>
    10	  <body>
    11	    <h1>Thoughts of Today </h1>
    12	    <div id = "inputField">
    13	      <textarea autofocus class = "entries" id = "entry1" placeholder="anything you'd like to dump"></textarea>
    14	          <p> input2</p>
    15
    16
    17	    </div>
    18
    19	  </body>
    20	</html>
```

11:06 AM

The error on line 10 comes from a missing closing brace } for the if block. In main.js:8-10, if(e.key === "Enter"){ opens a brace that never gets closed. When the parser reaches the) on line 10, it's still inside the if block, so it throws SyntaxError: Unexpected token ')'.

Fix:

```
input.addEventListener("keypress",(e)=>{
   if(e.key === "Enter"){
      userInput.push(input.value);
   }
})
```

Once that's fixed, you'll run into two more errors:

Line 5 selector: document.querySelector("entries") looks for an <entries> tag, which doesn't exist. Your textarea uses class="entries", so it needs a dot: document.querySelector(".entries").
Script timing: in index.html:7, main.js loads in the <head> before the textarea exists. That means input will be null, and input.addEventListener will fail with "Cannot read properties of null". Add defer to fix it: <script src="main.js" defer></script>.
The console.log(userInput) on line 12 also runs once when the page loads, so it will always print []. To see entries as you add them, move it inside the if block after the push.

main.js
why is my enter keypress things not working?
Bash Show current main.js and index.html
IN

```
cat -n main.js; cat -n index.html
```

OUT

```
     1  console.log("javescript linked");
     2
     3  window.onload = () => {
     4    let userInput = [];
     5
     6    let input = document.querySelector(".entries");
     7
     8    input.addEventListener("keypress", (e) => {
     9      if (e.key === "enter") {
    10        console.log("enterpressed");
    11        userInput.push(input.value);
    12        console.log(userInput);
    13      }
    14    });
    15  };
     1  <!doctype html>
     2  <html>
     3    <head>
     4      <meta charset="utf-8" />
     5      <title>ready.blog</title>
```

```
    <link rel="stylesheet" href="style.css" />
    <script src="main.js"></script>

  </head>
  <body>
    <h1>Thoughts of Today </h1>
    <div id = "inputField">
      <textarea autofocus class = "entries" id = "entry1" placeholder="anything you'd like to dump"></textarea>
          <p> input2</p>


    </div>

  </body>
</html>
```

11:10 AM

It's a capitalization issue. e.key is case-sensitive, and the Enter key reports as "Enter" with a capital E. In main.js:9, you're comparing against "enter", so the check never matches.

```
if (e.key === "Enter") {
```

Two other things you'll probably notice once it works:

Enter still adds a new line in the textarea. If you don't want that, call e.preventDefault(); inside the if. keypress is deprecated. It still fires for Enter, but keydown is the recommended event now and works the same way here:

```
input.addEventListener("keydown", (e) => {
  if (e.key === "Enter") {
    e.preventDefault();
    userInput.push(input.value);
    input.value = ""; // optional: clear the box after saving
    console.log(userInput);
  }
});
```

Your window.onload wrapper fixes the earlier timing problem, so the script can stay in the <head>.

style.css

So basically im building a web tool that can help me blog my project. Ideally i shouldn't need to swap between tabs or apps when I want to log some thought. i want to make blog it in the most casual way so that, not even need to worry about grammar.

the interface should be somewhat similar to the nutrition logging app mist. when the user press enter the page store the input and the cursor automatically jumps to the next text area. And when they are ready, they submit their entries and a AI will fix the grammar for them and put the entries into bulletpoint.

Right now I have my first textarea set up. I want cursor to jump to the next text area when i press enter
Show more
Bash Show style.css
IN
cat -n style.css

OUT

```
     1  p {
     2    font-family: sans-serif;
     3    color: red;
     4  }
     5
     6  #inputField{
     7  display: flex;
     8  flex-direction: column;
     9  }
    10
    11  .entries{
    12     border: none;
    13     outline: none;
    14
    15
    16  }
```

Write main.js
41 lines

```
console.log("javescript linked");

window.onload = () => {
  let inputField = document.querySelector("#inputField");

  // listen on the container so new textareas get the Enter behavior too
  inputField.addEventListener("keydown", (e) => {
    if (e.key === "Enter" && !e.shiftKey && e.target.classList.contains("entries")) {
      e.preventDefault(); // stop Enter from adding a new line

      let current = e.target;
      if (current.value.trim() === "") return; // don't make a new box for an empty entry

      // if there's already a textarea after this one, just jump to it
      let next = current.nextElementSibling;
      if (next && next.classList.contains("entries")) {
        next.focus();
        return;
```

```
    }

    // otherwise make a new textarea right after the current one
    let newEntry = document.createElement("textarea");
    newEntry.classList.add("entries");
    newEntry.placeholder = "anything you'd like to dump";
    current.after(newEntry);
    newEntry.focus();

    console.log(getEntries());
  }
});

// grab every non-empty entry, ready for submitting later
function getEntries() {
  let entries = [];
  document.querySelectorAll(".entries").forEach((box) => {
    if (box.value.trim() !== "") entries.push(box.value.trim());
  });
  return entries;
}
};
```

11:20 AM

I rewrote main.js so Enter now saves the entry and moves the cursor to a new textarea. I haven't opened it in a browser, so give it a quick try.

How it works:

One listener on #inputField: it's attached to the container rather than the first textarea. Textareas you add later get the same Enter behavior without needing their own listeners.

Pressing Enter:

It stops Enter from typing a line break.

It skips empty entries, so pressing Enter repeatedly doesn't stack up blank boxes.

If there's already a textarea right after the current one (for example, you clicked back up to edit an earlier entry), the cursor jumps to it. Otherwise it creates a new textarea below and focuses it.

Shift+Enter still adds a normal line break within an entry.

getEntries() replaces the userInput array. It reads every non-empty textarea when called, so edits to earlier entries are included. Your submit button can call it later and send the result to the AI. For now it logs the list to the console on each Enter so you can check it.

New textareas are inserted directly after the one you're typing in, which puts them above the input2 in index.html:14. If that paragraph was just a placeholder, you can delete it.

main.js

do not write anything for me, just tell me what concept to use. Please. Also when they hit enter, next textarea element should be created

12:28 PM
No response requested.

main.js
do not write anything for me, just tell me what concept to use. Please reverse to what I had. Also when they hit enter, the next textarea element should be created. Is the query selector the right thing to use?

Put what i need to do into bulletpoint
Show more
12:29 PM
Your main.js is already back to your own version, so I didn't change anything.

Is querySelector the right thing to use?

It's fine for grabbing the first textarea when the page loads.
The catch is that it returns only the first element that matches, and only the elements that exist at the moment it runs. Your listener is attached to that one textarea, so any textareas you create later won't respond to Enter.
querySelectorAll(".entries") won't solve this either. It also only finds what's on the page at the moment it runs.
What you need to do:

Use keydown instead of keypress. keypress is deprecated.
Stop Enter from typing a line break. Look up e.preventDefault().
Create the new textarea in JS. Look up document.createElement(), then give it the same class with classList.add().
Put it on the page. Look up appendChild() (adds to the end of a parent like #inputField) or element.after() (inserts right after a specific element).
Move the cursor into it. Look up the .focus() method.
Make new textareas respond to Enter too. You have two options:
Add the listener to each new textarea when you create it, or
Use event delegation: put one listener on the parent #inputField and use e.target to find which textarea was typed in. Search "event delegation JavaScript" or "event bubbling".
Know which textarea you're in. e.target gives you the textarea where Enter was pressed, so you can read its .value. Right now input.value always reads the first box.
Collecting entries for submit (later): you can keep pushing to your array, or loop over querySelectorAll(".entries") when the user submits. The loop also picks up any edits made to earlier entries.

main.js
with e.preventDefault, I cant type anything
Bash Show current main.js
IN
cat -n /Users/joychang/Documents/web-tool-fa26/midterm/main.js

OUT

```
1   console.log("javescript linked");
2
3   window.onload = () => {
4     let userInput = [];
5
6     let input = document.querySelector(".entries");
7
8     input.addEventListener("keydown", (e) => {
9       e.preventDefault();
10      if (e.key === "Enter") {
11        console.log(e);
12        userInput.push(input.value);
13        console.log(userInput);
14      }
15    });
16  };
```

4:42 PM

e.preventDefault() cancels whatever a key would normally do. In main.js:9 it sits outside your if, so it runs on every keydown and blocks every letter you type, not just Enter.

Move it inside the if (e.key === "Enter") block. Then it only cancels the Enter key's line break, and all other keys type normally.

The idea to remember: put code inside a condition when it should only run in that case. Anything outside the if runs on every key press.

main.js

what's wrong wuth what i had now:

main.js:21 Uncaught TypeError: Failed to execute 'appendChild' on 'Node': parameter 1 is not of type 'Node'.
    at HTMLTextAreaElement.<anonymous> (main.js:21:18)

Bash Show current main.js

IN

```
cat -n /Users/joychang/Documents/web-tool-fa26/midterm/main.js
```

OUT

```
1   console.log("javescript linked");
2
3   window.onload = () => {
4     let userInput = [];
5
6     let inputField = document.getElementById("inputField")
7     let input = document.getElementsByClassName("entries")[0];
8
9     input.addEventListener("keydown", (e) => {
10      if (e.key === "Enter") {
11        e.preventDefault();
```

```
12
13          console.log(e);
14          userInput.push(input.value);
15          console.log(userInput);
16          let newTextarea = document.createElement("textarea");
17          newTextarea.id = "entry2";
18          newTextarea.class = "entries";
19          newTextarea = "type yout thought here"
20
21          inputField.appendChild(newTextarea);
22
23        }
24      });
25    };
```

6:46 PM

The error comes from main.js:19. newTextarea = "type yout thought here" replaces the whole variable with a string. By line 21 newTextarea is no longer a textarea element, and appendChild() only accepts elements (nodes), so it throws.

Line 19: set a property on the textarea instead of reassigning the variable. If that text is supposed to be the gray hint, look up the .placeholder property.
Line 18: .class isn't a real property, so the class never gets added (no error, it just silently fails). Look up className or classList.add(). You want the class to apply so your CSS styles the new box.
Line 17: every Enter will create another textarea with id="entry2", and ids are supposed to be unique. Either drop the id (the class is enough) or build it from a counter.
Still missing:
.focus() on the new textarea, so the cursor moves into it.
Enter won't work inside the new textareas yet, because your listener is attached only to the first one.
That's the "listener on each new textarea" vs. "event delegation" choice from before.

main.js#9-9
what is wrong with my code now, it is not creating 3rd text area
Bash Show current main.js
IN
cat -n /Users/joychang/Documents/web-tool-fa26/midterm/main.js

OUT

```
1     console.log("javescript linked");
2
3     window.onload = () => {
4       let userInput = [];
5       let inputCount = 0;
6       let inputField = document.getElementById("inputField");
7       let input = document.getElementsByClassName("entries")[inputCount];
8
9       input.addEventListener("keydown", (e) => {
```

```
10        if (e.key === "Enter") {
11          e.preventDefault();
12
13          inputCount++;
14
15          console.log(e);
16          userInput.push(input.value);
17          console.log(userInput);
18          let newTextarea = document.createElement("textarea");
19          let idName = "entry" + inputCount;
20          newTextarea.id = idName;
21          newTextarea.classList.add("entries");
22          newTextarea.placeholder = "type yout thought here";
23
24          inputField.appendChild(newTextarea);
25          input = document.getElementsByClassName("entries")[inputCount];
26          input.focus();
27
28          console.log(inputCount);
29        }
30      });
31    };
```

7:07 PM

The listener stays on the first textarea. Changing the input variable doesn't move it.

main.js:9 runs once, when the page loads, and attaches the listener to whatever input is at that moment: the first textarea.

On line 25, you point input at the new textarea. That gives the variable a new value, but the listener is still attached to the original element.

So pressing Enter in the 2nd textarea does nothing. Pressing Enter in the 1st one still adds boxes (and line 16 saves input.value, which by then is the latest box, not the one you typed in).

The concept: a listener is attached to an element, not to a variable. Two ways to fix it:

Add a listener to each new textarea when you create it. Wrap the Enter logic in a named function so you can reuse it with addEventListener on every new box.

Event delegation: put one listener on inputField (the parent) and use e.target to get the textarea Enter was pressed in. This also gets rid of the need for input, inputCount, and getElementsByClassName(...)[inputCount].

One more thing: inputCount becomes 1 on the first Enter, so the new box gets id="entry1". That's the same id as the textarea in your HTML. Start the counter at 1, or drop the ids.

main.js

Event delegation: put one listener on inputField (the parent) and use e.target to get the textarea Enter was pressed in. This also gets rid of the need for input, inputCount, and getElementsByClassName(...)[inputCount].

how?
Show more
7:09 PM
Event delegation relies on event bubbling. When you press a key in a textarea, the keydown event fires on that textarea first, then "bubbles up" to its parent (#inputField), then to body, and so on. A listener on the parent hears key presses from every child inside it, including children created later.

How to apply it to your code:

Move the listener: call addEventListener("keydown", ...) on inputField instead of on input.
Find which textarea was typed in: inside the listener, e.target is the exact element where the key was pressed. Use it wherever you currently use input, both to read .value and as the box you're working from.
Check that it's a textarea: the parent might contain other elements. Add a check that e.target has the entries class (look up classList.contains()) in addition to checking for Enter.
Delete what you no longer need: the input variable, the getElementsByClassName(...)[inputCount] lines, and the reassignment on line 25. Focus the new textarea directly with the variable you just created it in.
Keep everything else: preventDefault, createElement, classList.add, placeholder, appendChild, focus.
A small example on a different element, to show the idea:

```
// one listener on the <ul> handles clicks on every <li>, even ones added later
list.addEventListener("click", (e) => {
  console.log(e.target.textContent); // the specific <li> that was clicked
});
```

To check your understanding, add console.log(e.target) inside your listener and press Enter in different boxes. You should see a different textarea logged each time.

main.js
.focus is not working
Bash Show current main.js
IN
cat -n /Users/joychang/Documents/web-tool-fa26/midterm/main.js

OUT

```
     1  console.log("javescript linked");
     2
     3  window.onload = () => {
     4    let userInput = [];
     5    let inputCount = 1;
     6    let inputField = document.getElementById("inputField");
     7    let input = document.getElementsByClassName("entries")[inputCount];
     8
     9    inputField.addEventListener("keydown", (e) => {
    10      if (e.key === "Enter") {
```

```
11          e.preventDefault();
12          console.log(e.target);
13
14          console.log(e);
15          userInput.push(e.target.value);
16          console.log(userInput);
17
18          inputCount++;
19          console.log(inputCount);
20
21          let newTextarea = document.createElement("textarea");
22          let idName = "entry" + inputCount;
23          newTextarea.id = idName;
24          newTextarea.classList.add("entries");
25          newTextarea.placeholder = "type yout thought here";
26
27          console.log(newTextarea)
28
29          inputField.appendChild(newTextarea);
30          input = document.getElementById("idName");
31          input.focus();
32        }
33      });
34    };
```

7:38 PM

The problem is on main.js:30: getElementById("idName") has quotes around idName.

Quotes make it a literal string. The browser searches for an element whose id is literally idName, finds nothing, and returns null. Line 31 then fails on null.focus(). You'll see a "Cannot read properties of null" error in the console.

Without quotes it would work, because idName is your variable holding "entry2", "entry3", and so on.

You don't need to look it up at all. You already have the element in newTextarea, so you can call .focus() on that directly. Lines 30 and 7, and the input variable, can all go.

The concept here is the difference between a variable name and a string. idName means the value stored in that variable, while "idName" is just that text.

main.js

okay now i have a general input/storing system, if i want to use chatgpt api to help me fix grammar, fix my wording, how would i do? Please check my code right now and tell me where/what to add (or fix slightly) do not change my code. just tell me

Bash Show index.html, git remote, and midterm files

IN

```
cat -n midterm/index.html; git remote -v; ls midterm
```

OUT

```
1   <!doctype html>
2   <html>
3     <head>
4       <meta charset="utf-8" />
5       <title>ready.blog</title>
6       <link rel="stylesheet" href="style.css" />
7       <script src="main.js"></script>
8     </head>
9     <body>
10      <h1>Thoughts of Today</h1>
11      <div id="inputField">
12        <textarea
13          autofocus
14          class="entries"
15          id="entry1"
16          placeholder="anything you'd like to dump"
17        ></textarea>
18      </div>
19      <div id = "submit">
20        <button class="button" id="submitButton">submit</button>
21      </div>
22
23      <div id = "outputField">
24        <p id = "output"> </p>
25      </div>
26    </body>
27  </html>
origin  https://github.com/tc4416/web-tool-fa26.git (fetch)
origin  https://github.com/tc4416/web-tool-fa26.git (push)
index.html
main.js
style.css
```

7:39 PM

Your repo is pushed to a public GitHub repo, so start with the API key.

1. Don't put your API key in main.js.

Anything in front-end JavaScript is visible to anyone who opens the page or the repo. Bots scan GitHub for leaked OpenAI keys within minutes and run up charges on them.

So you need a small server that sits between your page and OpenAI:

The page sends your entries to your server.

The server adds the secret key and calls OpenAI.

The server sends the result back to the page.

GitHub Pages can't run server code. Options to look up:

Node.js + Express running on your laptop. Simplest for a class project.

A serverless function (Vercel, Netlify Functions, or Cloudflare Workers) if you want it online.

Keep the key in an environment variable in a .env file, and add .env to .gitignore so it never gets committed.
2. Fix these in your current code first:

Line 7: input isn't needed anymore. You can delete it.

Line 37: getElementById(idName) works, but newTextarea.focus() does the same thing without the lookup.

Line 22: userInput saves each entry when you press Enter, so:

If you go back and edit an earlier box, the array keeps the old text.
The last box is never saved unless you press Enter before clicking submit.
Pressing Enter on an empty box saves an empty string.
Collecting the entries when submit is clicked fixes all three: loop over querySelectorAll(".entries"), skip empty ones (look up .trim()), and build the array there.

Lines 44–47: innerText += adds to whatever is already there, so clicking submit twice shows the list twice. Clear output at the start of the click handler.

Lines 12–14: the empty Shift+Enter branch works, but you could just check e.key === "Enter" && !e.shiftKey in a single if.

3. What to add in main.js, inside the submit listener (line 42):

Make the callback an async function, so you can wait for the AI's reply.
Use fetch() to send a POST request to your own server's URL (not OpenAI's):
Turn your entries array into JSON with JSON.stringify() for the body.
Set a Content-Type: application/json header.
await the response, then turn it back into data with .json().
Put the AI's result into output instead of (or in addition to) your raw entries.
Show a "fixing…" message while waiting, since the request takes a few seconds.
Wrap the request in try/catch so a failed request shows an error instead of failing silently.
4. What the server does:

It has one route (for example POST /fix) that receives the entries.
It calls OpenAI's API with your key from the environment variable. Look up the official openai npm package, and check OpenAI's docs for the current model names.
It sends the AI's text back to the page as JSON.
During local development, the page and the server will be on different ports, which the browser blocks by default. Look up CORS.
5. Writing the prompt (this is where you control how the result reads):

Give the AI a system/instructions message. For example: fix grammar and wording, keep my meaning and casual tone, don't add new ideas, return one bullet point per entry.
Send your entries as the user message.

If you want to put each bullet into its own HTML element, ask for the result as a JSON array instead of plain text. That's easier to loop over than splitting a string.
Concepts to look up, in order: async/await, fetch with POST, JSON.stringify/.json(), Express routes, environment variables + .gitignore, CORS, OpenAI chat API (messages with roles).

ollama.html#11-11
can you get me a step-by-step guide on how to use the openAI api?

https://developers.openai.com/api/docs/quickstart

I dont think i need to worry about safety for now about server and stuff.
Show more
Bash Show the class Ollama demo and gitignore
IN
ls demo1006 && cat -n demo1006/ollama.html; cat .gitignore 2>/dev/null

OUT
Exit code 1
ollama.html

```
     1  <!doctype html>
     2
     3  <header></header>
     4
     5  <body>
     6    <!-- <h1>http://itp-ml.itp.tsoa.nyu.edu:11434/</h1> -->
     7    <input id="webPageDescription" />ask question/>
     8    <button id="startButton">start</button>
     9
    10    <script>
    11      let url = "http://itp-ml.itp.tsoa.nyu.edu:11434/api/chat";
    12
    13      async function queryLanguageModel(userQuery) {
    14        let ollamaOptions = {
    15          model: "gemma4:e4b",
    16          messages: [
    17            {
    18              role: "user",
    19              content: "what do you think about the intersection between fortune telling/divonation vs predictive text/algorithm in modern technologies? i would want my thesis (@nyu itp ) to be related to this topic. WHat can my argument be?",
    20            },
    21          ],
    22          // messages: [
    23          //   {
    24          //     role: "user",
```

```
25            //     content: "what do you think about" + userQuery,
26            //   },
27            // ],
28            stream: false,
29          };
30
31          let options = {
32            method: "POST",
33            body: JSON.stringify(ollamaOptions),
34          };
35
36          let response = await fetch(url, options);
37          let data = await response.json();
38          let modelResponse = data.message.content;
39          console.log(modelResponse);
40
41          //   document.body.innerHTML= modelResponse
42          //
43        }
44
45        queryLanguageModel();
46        let button = document.getElementById("startButton");
47        button.addEventListener("click", function () {
48          let myInput = dovumrny.getElementByID("webPageDescription");
49          console.log(myInput.value);
50          queryLanguageModel(myInput.value);
51        });
52      </script>
53    </body>
```

Web Fetchhttps://developers.openai.com/api/docs/quickstart
Fetched from https://developers.openai.com/api/docs/quickstart
7:54 PM
Here's how to use the OpenAI API from your page. You already have almost all of it: your class demo ollama.html uses the same fetch pattern, so switching to OpenAI only changes four things.

Step 1: Get an API key

Sign in at platform.openai.com, add billing (the API isn't free; a few dollars of credit is plenty), and create a key under API keys.
Copy it right away. You can only see it once.
Set a low monthly spend limit in billing settings in case the key leaks.
Step 2: Keep the key out of GitHub (two minutes, worth it)

Put the key in its own file, like midterm/config.js, holding just one variable with the key in it.
Add that filename to a .gitignore file at the repo root. Your repo doesn't have one yet.
Load config.js with its own <script> tag before main.js in index.html.
If you skip this and push, bots will grab the key, and GitHub may block the push anyway.

Step 3: Copy the fetch pattern from ollama.html into your submit handler
Only these parts change:

Ollama (class demo) OpenAI
URL itp-ml...:11434/api/chat https://api.openai.com/v1/responses
Headers none Content-Type: application/json and Authorization: Bearer <your key>
Body model, messages, stream model (the docs use "gpt-6-astra"), instructions (your rules), input (your entries as text)
Where the answer is data.message.content inside data.output, see Step 5
headers is a new key in your options object, next to method and body.
Make the submit callback async so you can await the fetch, just like queryLanguageModel does.
Step 4: Build the request from your entries

instructions: a string telling the AI what to do. For example: "Fix grammar and wording, keep my meaning and casual tone, don't add ideas, return one bullet point per entry."
input: your userInput array joined into one string. Look up .join("\n").
Step 5: Find the answer in the response

Add console.log(data) and expand it in the browser console.
The text is in data.output: find the item with type: "message", then look in its content[0].text.
Some models put a "reasoning" item first, so the message isn't always output[0]. Check the console rather than guessing the index.
The docs mention response.output_text, but that only exists in their npm package, not in a plain fetch response.
Step 6: Show it

Put that text into your output element instead of looping over userInput.
Since your prompt asks for bullets, the AI already formats them, so you can drop your own "- " loop.
If it breaks:

401: the key is wrong or missing from the Authorization header. Check for the space after Bearer.
429: no billing credit on the account.
400: the body is malformed. The error message in data says which field.
A no-key option: your class Ollama server works the same way with no key and no billing, and you've already used it. Just change your instructions and input in messages. It probably only works on NYU wifi or VPN, so test it before deciding.

style.css
i want the submit button be in the middle
Bash Show CSS and submit button markup
IN
cd /Users/joychang/Documents/web-tool-fa26/midterm && cat -n style.css; grep -n -i -A2 "submit" index.html

OUT

```
1     p {
2       font-family: sans-serif;
3       color: rgb(25, 25, 25);
4     }
5     h1{
6         font-family: sans-serif;
7         color: rgb(56, 55, 55);
8     }
9
10    #inputField{
11    display: flex;
12    flex-direction: column;
13    }
14
15    .entries{
16        border: none;
17        outline: none;
18    }22:    <div id = "submit">
23:     <button class="button" id="submitButton">submit</button>
24-   </div>
25-
```

12:35 AM

Style the button's parent, , rather than the button itself. The button only takes up as much width as its text, so centering means centering it inside the div, which spans the full width of the page. There are no rules for #submit in style.css yet, so add a new block for it.

You can use either approach:

text-align: center on #submit. Buttons are inline elements, so they follow text alignment just like words do. This is the simplest option.
Flexbox on #submit: display: flex plus justify-content: center. It's the same idea you already used for #inputField. justify-content controls where items sit along the row, and center puts them in the middle.
If you meant the middle of the screen vertically too, that's a different setup. Look up align-items: center combined with a height on the container.

main.js

i want the interrface to look like I am writing down bullet point. Please help me add a bullet point dot in front of entry, and Also I want the output to be bullet point line by line. It should change line and be ready to copy and paste to my blog

Bash Check whether the API key was committed or pushed

IN

```
git status --short midterm; git log --oneline -5 -- midterm; git fetch -q 2>/dev/null; git log --oneline origin/main -3 -- midterm; git log -p --all -S "sk-proj-GoVCr" --oneline -- midterm | head -5; cat .gitignore 2>/dev/null; ls midterm
```

OUT
?? midterm/
config.js
icon.png
index.html
main.js
node_modules
package-lock.json
package.json
style.css
Bash Check config.js, script tags, and CSS
IN
cd /Users/joychang/Documents/web-tool-fa26/midterm && cat config.js | sed -E
's/sk-[A-Za-z0-9_-]+/sk-REDACTED/g'; grep -n "script" index.html; cat -n style.css

OUT
key = "sk-REDACTED"7:   <script src = "config.js"></script>
8:    <script src="main.js"></script>
```
     1	p {
     2	  font-family: sans-serif;
     3	  color: rgb(25, 25, 25);
     4	}
     5	h1{
     6	    font-family: sans-serif;
     7	    color: rgb(56, 55, 55);
     8	}
     9
    10	#inputField{
    11	display: flex;
    12	flex-direction: column;
    13	}
    14
    15	.entries{
    16	    border: none;
    17	    outline: none;
    18	}
    19
    20	#submit{
    21	    text-align: center;
    22	}
```
12:36 AM
Before your next commit: your real API key is typed directly into main.js:14. The midterm/ folder has never been committed, so it hasn't leaked yet.

Use the key from config.js instead. It already defines a key variable, and index.html loads it before main.js. Set apiKey to that variable instead of pasting the key into main.js.
Create a .gitignore at the repo root listing midterm/config.js and node_modules, before you run git add.

If the key ever does get pushed, revoke it on the OpenAI dashboard right away.
1. Bullet dot in front of each entry

The usual CSS trick won't work here. ::before adds a symbol before an element's content, but it does nothing on a <textarea>, because the browser draws textareas itself.
Wrap each textarea in a list item instead. HTML lists draw their own bullets:
In index.html, make #inputField a <ul> and put the first textarea inside an <li>.
In your Enter handler, create an <li> with createElement, put newTextarea inside it with appendChild, then append the <li> to inputField.
Your event delegation still works. e.target is still the textarea, because the event bubbles up through the <li> to the <ul>.
Remove display: flex from #inputField. Flex can hide list bullets.
Look up list-style to change the bullet style, and padding-left on the <ul> to adjust spacing.
2. Output that goes line by line

Why it's on one line now: on line 58 you set output.textContent. textContent keeps the \n line breaks in the text, but HTML displays them as spaces, so it all shows as one line. (Line 93 uses innerText, which does turn \n into real line breaks, which is why that version looked fine.)
Fix in CSS: add white-space: pre-wrap to #output. That tells the browser to show line breaks exactly as they appear in the text. This also matters for copying: when you select and copy the output, the line breaks come with it.
Line 7 / line 83: you can call .focus() directly on newTextarea; input isn't needed.
3. Ready to paste into your blog

Pick the bullet format your blog understands:

If it supports Markdown (GitHub, Notion, many blog platforms), keep - at the start of each line. The blog will turn it into a real list.
If it's plain text, ask for • instead.
Either way, say so in the system prompt, e.g. "Start each line with - , one bullet per line, no blank lines."
Right now the AI is guessing.

Optional "copy" button: look up navigator.clipboard.writeText() and pass it output.textContent. One click and the text is on your clipboard, with no selecting needed.

index.html
The usual CSS trick won't work here. ::before adds a symbol before an element's content, but it does nothing on a <textarea>, because the browser draws textareas itself.
Wrap each textarea in a list item instead. HTML lists draw their own bullets:
In index.html, make #inputField a <ul> and put the first textarea inside an <li>.
In your Enter handler, create an <li> with createElement, put newTextarea inside it with appendChild, then append the <li> to inputField.
Your event delegation still works. e.target is still the textarea, because the event bubbles up through the <li> to the <ul>.
Remove display: flex from #inputField. Flex can hide list bullets.

Look up list-style to change the bullet style, and padding-left on the <ul> to adjust spacing.
show me code, do not change it for me though
Show more
Bash Show current HTML, CSS, and Enter handler
IN
cat -n index.html; cat -n style.css; sed -n 55,100p main.js | cat -n | sed 's/^/ /'

OUT

```
     1  <!doctype html>
     2  <html>
     3    <head>
     4      <meta charset="utf-8" />
     5      <title>ready.blog</title>
     6      <link rel="stylesheet" href="style.css" />
     7      <script src = "config.js"></script>
     8      <script src="main.js"></script>
     9          <link rel="icon" type="image/x-icon" href="icon.png">
    10
    11    </head>
    12    <body>
    13      <h1>Thoughts of Today</h1>
    14      <div id="inputField">
    15        <ul>
    16          <li>
    17        <textarea
    18          autofocus
    19          class="entries"
    20          id="entry1"
    21          placeholder="anything you'd like to dump"
    22        ></textarea>
    23        </li>
    24        </ul>
    25      </div>
    26      <div id = "submit">
    27        <button class="button" id="submitButton">submit</button>
    28      </div>
    29
    30      <div id = "outputField">
    31        <p id = "output"> </p>
    32      </div>
    33    </body>
    34  </html>
     1  p {
     2    font-family: sans-serif;
     3    color: rgb(25, 25, 25);
     4  }
     5  h1{
```

```
    font-family: sans-serif;
    color: rgb(56, 55, 55);
}

#inputField{
display: flex;
flex-direction: column;
}

.entries{
    border: none;
    outline: none;
}

#submit{
    text-align: center;
}       1
    let modelResponse = data.choices[0].message.content;
    console.log(modelResponse);
    output.textContent = modelResponse;
  }

  inputField.addEventListener("keydown", (e) => {
    if (e.key == "Enter" && e.shiftKey) {
    } else if (e.key === "Enter") {
      e.preventDefault();
      console.log(e.target);

      console.log(e);
      userInput.push(e.target.value);
      console.log(userInput);

      inputCount++;
      console.log(inputCount);

      let newTextarea = document.createElement("textarea");
      let idName = "entry" + inputCount;
      newTextarea.id = idName;
      newTextarea.classList.add("entries");
      newTextarea.placeholder = "type yout thought here";

      console.log(newTextarea);

      inputField.appendChild(newTextarea);
      input = document.getElementById(idName);
      input.focus();
    }
```

```
    });

    submit.addEventListener("click", () => {
      let bullets = "";
      for (let i = 0; i < userInput.length; i++) {
        bullets += "- " + userInput[i] + "\n";
      }
      output.innerText = bullets;
      queryLanguageModel(bullets);
    });
  };
```

12:43 AM

You've already done the HTML part: index.html:14-25 has the <ul> and <li> inside #inputField. What's left is in your JS and CSS. Here's your current code with comments marking each spot. I haven't changed any files.

main.js:74-84, inside your Enter handler:

```
    let newTextarea = document.createElement("textarea");
    // ← ADD: also create an <li> here with createElement

    let idName = "entry" + inputCount;
    newTextarea.id = idName;
    newTextarea.classList.add("entries");
    newTextarea.placeholder = "type yout thought here";

    console.log(newTextarea);

    inputField.appendChild(newTextarea);
    // ← CHANGE: two problems on this line:
    //   1. newTextarea should go INTO the <li>, not straight onto the page
    //   2. inputField is your <div>; new <li>s belong inside the <ul>
    //      → grab the <ul> near the top (next to line 6) and append the <li> to it

    input = document.getElementById(idName);
    input.focus();
    // ← OPTIONAL: newTextarea.focus() does the same thing
```

Right now new textareas get added to the , after the <ul> closes. That's why only the first entry has a bullet.

main.js:6:

```
  let inputField = document.getElementById("inputField");
  // ← keep this for your keydown listener (delegation still works on the div)
  // ← ADD a variable for the <ul> below it. Your <ul> has no id, so either give it
```

// one in index.html or select it with querySelector("#inputField ul")
style.css:10-13:

#inputField{
display: flex; /* ← REMOVE these two lines; flex on the div */
flex-direction: column; /* ← isn't needed now that the list stacks entries */
}
Optional: add a rule for #inputField ul with padding-left to move the bullets closer to the edge or further from it.

style.css:15-18, optional: you'll probably want width on .entries so each box stretches across the line like a real bullet point.